

AI-Powered AUTOSAR HLD Document Analysis Assistant using Retrieval-Augmented Generation

Technical Report - Case Study CS1

Student	Diksha
PRN	123B1B249
Programme / Batch	B.Tech Computer Engineering, 2023-2027
Institute	Pimpri Chinchwad College of Engineering (PCCOE), Pune
Case Study	CS1 - AUTOSAR HLD Document Analysis Assistant
Report date	2026-10-06

1. Case Study and Project Title

Case Study CS1 from the Automotive Engineering AI case-study document (v1.0, September 2026): an assistant that extracts, organises, searches and validates architectural knowledge from long, unstructured AUTOSAR High-Level Design (HLD) PDFs. Engineers ask questions in plain English and receive evidence-based answers with page and section citations; a human reviews every output.

2. Objectives

- Reduce manual effort in reviewing HLD documents through cited question answering.
- Create a structured, searchable repository of components, ports, interfaces, signals, dependencies and requirements.
- Improve traceability and flag potential inconsistencies and missing dependencies.
- Support design review, onboarding and impact analysis, including comparison of document revisions.

3. Scope

Implemented: controlled PDF ingestion; heading, table and metadata extraction; section-aware chunking; entity extraction; hybrid RAG with citations; relevance gate and abstention; deterministic KB tools; consistency checks R1-R7; revision comparison; CSV/JSON export; human review with audit trail; FastAPI + Streamlit; Docker; evaluation suite.

Not implemented (by design): automatic approval of architecture decisions; modification of source HLDs; parsing of diagrams; enterprise IAM; graph database; use of real or copyrighted OEM / standards documents. OCR for scanned pages exists as an optional hook (requires Tesseract) and was **not evaluated**.

4. Proposed Approach and Implementation

4.1 Models and stack

Component	Choice	Notes
LLM	Qwen2.5-7B-Instruct via Ollama (local); configurable	Temperature 0; answers only from numbered sources; falls back to extractive generator if unavailable
Embeddings	BAAI/bge-small-en-v1.5 (sentence-transformers)	Offline hashed-token embedder used only if BGE cannot load
Vector store	ChromaDB (persistent, cosine)	Collection per project + embedder; metadata filters by document version
Lexical index	BM25 (rank-bm25), identifier-aware tokens	Fused with dense results by Reciprocal Rank Fusion (k=60)
Service / UI	FastAPI, Streamlit, SQLite	API-key roles (engineer / viewer); reviews and audit log in SQLite
Packaging	Docker, docker-compose, pytest	Config hash and data hashes logged per evaluation run

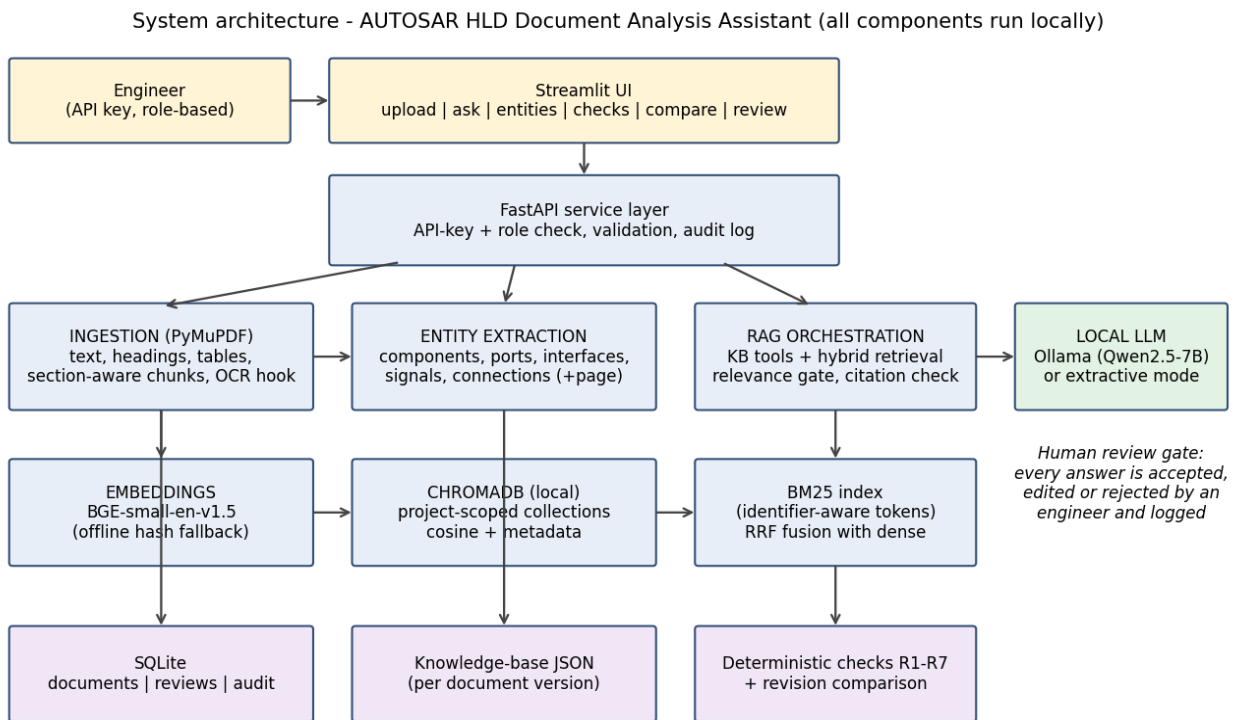


Figure 1: System architecture

4.2 Pipeline

- **Ingestion:** PyMuPDF reads text blocks, detects headings by font size, removes footers, and extracts tables (cell text is read by clipping, which keeps identifier underscores intact). Text becomes section-aware chunks (max 900 characters, one-sentence overlap); every table row becomes its own chunk.
- **Entity extraction:** typed tables are parsed into components, ports, interfaces, signals, connections, constraints and DIDs, each with page and section provenance, saved as a versioned knowledge base (JSON).
- **Retrieval:** dense (ChromaDB) and BM25 candidates are fused with RRF; top-6 chunks are returned. A *relevance gate* measures the share of query content terms found in the evidence and abstains below 0.6.
- **Deterministic KB tools:** questions about ports, dependencies, provider / consumer of a signal, CAN message fields and interfaces are answered from the extracted tables and passed to the generator as extra cited sources.
- **Generation and validation:** the LLM must cite every sentence as [S#]; unknown ids are stripped, groundedness and confidence are computed, and the response is marked for engineer review. Source text is treated as data, not instructions (prompt-injection control).
- **Checks and comparison:** rules R1-R7 (unconnected required port, signal length vs data type, undefined component, near-duplicate names, interface mismatch, undefined references, unused provided port) and entity / section diff between document versions.

5. Input Data / Knowledge Base

Only **synthetic** data is used. *BDC_HLD_v1.0.pdf* and *BDC_HLD_v1.1.pdf* describe a fictitious Body Domain Controller (7-8 software components, 17-19 ports, 5-6 CAN signals, 8-10 connections, about 7 pages each) and were generated by `Code/scripts/generate_synthetic_hld.py`. Four defects are seeded in v1.0 (unconnected required port, 8-bit signal vs uint16 data type, reference to an undefined component, DoorLockState vs DoorLockStatus) and two remain in v1.1. Revision v1.1 adds a component, raises the auto-lock threshold from 15 to 20 km/h and widens one signal, which gives 10 expected changes for revision comparison. Preprocessing and EDA: chunk counts (95 in v1.0, 105 in v1.1), table vs text chunk split, entity counts per category, and comparison with the ground-truth knowledge base.

6. Evaluation

Configuration of the reported run: embedder = hash1024, generation = extractive, retrieval = hybrid, top-k = 6, config hash 97aca1a26a19ba9f, data hash 22c9378bda37d997, run on 2026-10-06T18:26:22.

Important: this run used the offline fallback configuration (hashed lexical embeddings and/or extractive generation) because BGE and the local LLM could not be loaded in the build environment. Dense semantic retrieval and LLM answer quality are therefore **not** represented by these numbers. Re-run *python Code/evaluation/evaluate.py --embedder bge --llm ollama* and rebuild this report to obtain the final figures.

6.1 Method

Questions: 31 answerable + 5 out-of-scope (main set) and 15 answerable + 3 out-of-scope (**held-out**, written after the pipeline was tuned on the main set). Answer accuracy = all required key facts appear in the answer (strict substring proxy). Hit@k / MRR / evidence recall use gold evidence strings. Groundedness = share of answer sentences carrying a valid [S#]; citation correctness = a cited source contains the gold evidence. Extraction, consistency checks and revision comparison are scored against ground-truth files generated from the same data model as the PDFs.

6.2 Retrieval ablation (main set, answerable questions)

Mode	Hit@1	Hit@3	Hit@6	MRR	Evidence recall@6
bm25	0.645	0.935	0.968	0.798	0.968
dense	0.71	0.839	0.935	0.783	0.935
hybrid	0.742	0.903	0.968	0.821	0.968

6.3 End-to-end question answering

Metric	Main set	Held-out set
Questions	36	18
Answer accuracy (answerable)	1.0	0.8
Out-of-scope abstention rate	1.0	1.0
False abstention rate	0.0	0.133
Average groundedness	0.982	1.0
Citation validity rate	1.0	1.0
Citation correctness rate	1.0	0.8
Latency mean / p95 (s)	0.009 / 0.009	0.008 / 0.01

6.4 Extraction, consistency checks and revision comparison

Task	Precision	Recall	F1	Detail
Entity extraction (7 categories, 2 versions)	1.0	1.0	1.0	118/118 records exact match
Consistency checks v1.0	1.0	1.0	1.0	4 flagged, 4 seeded
Consistency checks v1.1	1.0	1.0	1.0	2 flagged, 2 seeded
Revision comparison v1.0 to v1.1	1.0	1.0	1.0	10/10 changes found

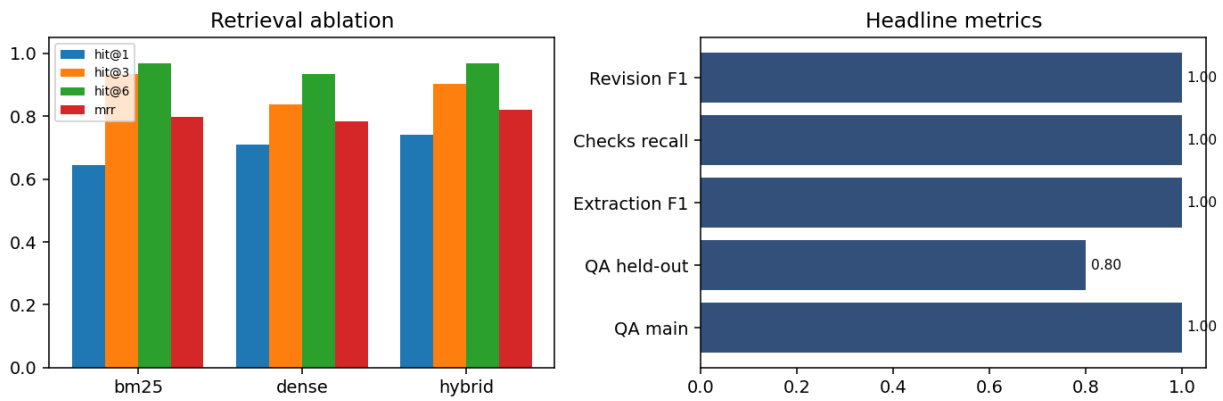


Figure 2: Retrieval ablation and headline metrics

6.5 Failure analysis

ID	Question	Outcome	Likely cause
H05	Does the lighting component need the speed signal?	incomplete answer	paraphrase with little lexical overlap with the document (e.g. checksum vs CRC, lamp command vs LampCmd)
H06	For how long is the last speed value kept when its checksum is wrong?	abstained	paraphrase with little lexical overlap with the document (e.g. checksum vs CRC, lamp command vs LampCmd)
H15	Which CAN message carries the lamp command and how often is it sent?	abstained	paraphrase with little lexical overlap with the document (e.g. checksum vs CRC, lamp command vs LampCmd)

The main-set scores are high partly because the pipeline was tuned on that set and the synthetic documents are clean and consistently formatted; the held-out set is the more honest estimate and still reflects an easy dataset compared with real, hundreds-of-pages HLDs.

7. Responsible AI, Security and MLOps

- Grounding: every source carries version, page and section; unknown citation ids are removed; unsupported questions get an explicit abstention.
- Human oversight: each answer is accepted, edited or rejected in the UI and recorded with the user, time and sources; the audit log records ingestion, queries and reviews.
- Privacy and security: local models and vector store, no runtime external API, PDF type and size validation, API-key roles, source text treated as untrusted data.
- MLOps: pinned requirements, Dockerfile and docker-compose (written but not executed in the build environment), pytest suite (8 tests), run log with config and data hashes.

8. Limitations and Future Work

- Synthetic, clean, single-column PDFs: real HLDs contain diagrams, merged table cells, scanned pages and inconsistent formatting. Diagram parsing is not supported; OCR is untested.
- Table typing relies on known column headers; other templates need new header mappings.
- The relevance gate is lexical, so heavily paraphrased questions can be wrongly refused (see failure analysis); BGE embeddings and a rewriting step should help.
- Accuracy uses substring matching; an LLM judge or manual grading would be fairer for free-text answers. The question sets are small (54 questions).
- Future work: revision graph and Neo4j traceability, role-based access per project, safety / cybersecurity integrations as in the case-study roadmap.

9. Student Contribution and AI-Tool Declaration

Claude (Anthropic) was used as a coding and documentation assistant for scaffolding, debugging, the synthetic-data generator, test / evaluation scripts and drafting of documents. The student reviewed, executed and is responsible for explaining all submitted artifacts. See Declarations/AI_Tool_Usage_Declaration.docx.

10. References

- Automotive Engineering AI - Standard Project Case Study Document, v1.0, September 2026 (Case Study 1).
- BAAI/bge-small-en-v1.5 model card; Ollama and Qwen2.5 documentation; ChromaDB, FastAPI, Streamlit, PyMuPDF documentation.
- Reciprocal Rank Fusion: Cormack, Clarke and Buettcher, SIGIR 2009.